ECE 408

第 11 讲

Computation in Convolutional Neural Networks

目录

1 卷积网络的结构与局部计算 **1**
1.1 从全连接计算到局部连接 . . . 1
1.2 参数共享与平移等变性 . . . 2
1.3 空间维度、通道与特征图 . . . 2
1.4 卷积、池化与分类层的组合 . . . 3

2 卷积与池化的前向计算 **4**
2.1 张量形状与求和维度 . . . 4
2.2 有效卷积的输出尺寸 . . . 5
2.3 步幅与零填充的尺寸关系 . . . 5
2.4 逐通道累加的完整算例 . . . 6
2.5 参数量、乘加次数与张量存储量 . . . 7
2.6 局部池化与非线性变换 . . . 7
2.7 池化的数值解释与独立性 . . . 8

3 直接卷积的并行实现与数据复用 **9**
3.1 张量地址与顺序参考实现 . . . 9
3.2 输出并行与归约维度 . . . 12
3.3 空间分块与线程索引 . . . 13
3.4 基础卷积核与边界处理 . . . 14
3.5 共享内存中的输入复用 . . . 16
3.6 协作加载、同步与权重复用 . . . 16

4 卷积的矩阵表示与输入展开 **18**
4.1 从局部内积到矩阵乘法 . . . 18
4.2 三通道、双输出的完整矩阵算例 . . . 19
4.3 展开索引与实际数组实现 . . . 20
4.4 批次组织与输出内存顺序 . . . 23
4.5 展开存储与矩阵分块复用 . . . 23
4.6 隐式展开与共享内存、寄存器联合分块 . . . 24

5 工程规模与综合例题 **26**
5.1 两层卷积的形状与运算规模 . . . 26
5.2 展开工作区与优化评估 . . . 27
5.3 三维卷积的存储读取比 . . . 28
5.4 多通道卷积的浮点运算总数 . . . 29
5.5 计算组织的统一关系 . . . 29

1 卷积网络的结构与局部计算

卷积神经网络（Convolutional Neural Network，CNN）用局部加权求和提取特征，再通过非线性变换和空间下采样构成多层计算。实现这类网络时，需要同时确定三个相互联系的对象：每个输出值依赖哪些输入，张量中的元素如何存放，以及哪些输出可以交给不同线程独立计算。卷积层的数学定义决定数据依赖，而线程映射和存储组织决定这些依赖如何在图形处理器（Graphics Processing Unit，GPU）上得到高效执行。

1.1 从全连接计算到局部连接

多层感知机（Multi-Layer Perceptron，MLP）的全连接层把输入写成向量 $\mathbf{x} \in \mathbb{R}^n$。若该层有 m 个输出，则第 j 个输出为 $a_j = \phi(\sum_{i=0}^{n-1} V_{j,i} x_i + \beta_j)$，其中 $V \in \mathbb{R}^{m \times n}$ 是权重矩阵，β_j 是偏置，ϕ 是逐元素激活函数。一个输出连接全部 n 个输入，因此该层有 mn 个权重和 m 个偏置。

例如，把一幅 28×28 的灰度图像展平为 784 维向量，先连接到 10 个隐藏神经元，再连接到 10 个输出神经元，总参数量为 $(784 \times 10 + 10) + (10 \times 10 + 10) = 7\,960$。当图像增大到 250×250，输入维数为 62 500；若隐藏层也有 62 500 个神经元，仅第一层就需要 $62\,500^2 = 3\,906\,250\,000$ 个权重。按每个权重占 4 字节计算，权重存储约为 15.625 GB，还没有计入偏置、激活值以及训练时的其他数组。

图像中的局部边缘、纹理等模式通常可以通过一个较小区域内的像素关系描述。卷积层因此让每个输出只连接输入中的局部窗口，形成稀疏交互（sparse interaction）。一个 $K \times K$ 的窗口在单个输入通道上只需要 K^2 次对应元素相乘，而不需要访问整幅图像。这个窗口是该输出相对于上一层的感受野（receptive field）。

局部连接与多层组合可以同时存在。在窗口完整落入有效输入范围的位置，若每层都使用步幅为 1 的 3×3 卷积，第一层输出依赖输入中的 3×3 区域；第二层把第一层相邻的 3×3 个位置组合起来，这些位置对应原输入中的 5×5 区域。继续堆叠 ℓ 层时，沿任一空间轴的感受野宽度为 $r_\ell = 1 + \ell(K-1)$。这个关系假设各层核宽均为 K、没有空洞且步幅均为 1；它说明局部运算可以逐层建立较大范围的数据依赖。

1.2 参数共享与平移等变性

卷积层还把同一组权重应用到不同空间位置，这称为参数共享（parameter sharing）。对于固定的输入通道和输出通道，窗口向右或向下移动时，卷积核系数保持不变。输出位置改变的是取用的输入数据，而不改变模型参数。若输入有 C 个通道、输出有 M 个通道、每个二维核为 $K\times K$，则权重总数为 MCK^2；每个输出通道再有一个偏置时，参数总数为 MCK^2+M。这里没有乘输出的空间位置数。

例如，单通道图像经过 16 个 5×5 核产生 16 张特征图，只需 $16\times 1\times 25=400$ 个权重，含偏置时为 416 个参数。只要输入仍有一个通道，这组权重就能作用于不同高度和宽度的图像，输出特征图的尺寸随之改变。若后续把全部空间位置展平再送入一个固定大小的全连接层，则该全连接层的输入维数仍然固定；因此，卷积核可复用于不同图像尺寸，与整个网络能否接收任意尺寸，是两个需要分别检查的条件。

参数共享还产生平移等变性（translation equivariance）：输入中的局部模式平移后，对应响应的位置随之平移。以单通道、步幅为 1 的二维卷积为例，记其算子为 $\mathcal{C}_F$，令 Δ_h,Δ_w 为整数。二维平移算子满足 $(T_\Delta X)[h,w]=X[h-\Delta_h,w-\Delta_w]$，卷积核不随位置改变，且所比较位置的取值和边界处理均有效，则

$$\mathcal{C}_F(T_\Delta X)[h,w]=\sum_{p,q}F[p,q]X[h+p-\Delta_h,w+q-\Delta_w]=(T_\Delta\mathcal{C}_F(X))[h,w]. \tag{1}$$

左右两边使用完全相同的乘积项，只是描述坐标不同。等变性保留特征出现的位置；如果需要输入平移后最终分类结果保持不变，还需要后续的汇聚和分类过程。有限图像的裁剪、填充方式，以及步幅大于 1 的下采样，会限制这种逐像素平移对应关系。

1.3 空间维度、通道与特征图

空间维度描述卷积窗口移动的方向，通道（channel）描述同一位置上并列的不同信号。一幅彩色图像有高度和宽度两个空间轴，在每个位置有红、绿、蓝三个颜色通道。三通道二维卷积的窗口沿两个空间轴移动，并在三个通道之间求和；它不会因为存在三个通道就变成沿三个空间轴移动的三维卷积。

输出通道也称输出特征图（feature map）。每个输出通道有一整组覆盖所有输入通道的权重：对各输入通道分别计算局部加权和，再把这些部分结果相加，得到一个输出值。因而 C 个输入通道与 M 个输出通道之间共有 $C\times M$ 个二维卷积核，但最终输出是 M 张二维特征图。不同的输出通道拥有不同权重，可以提取不同模式；同一输出通道的权重在所有图像和空间位置上共享。

表 1 空间轴与通道的不同含义

数据对象	卷积移动的轴	通道的含义
音频波形	一维时间轴	单路或多路音频信号
骨架动作序列	一维时间轴	不同关节的角度或坐标分量
彩色图像	二维高度、宽度	同一像素的红、绿、蓝分量
时频表示	二维时间、频率	同一时频位置的不同测量分量
体数据	三维空间坐标	同一体素的一个或多个测量分量
彩色视频	二维空间与一维时间	同一时空位置的颜色分量

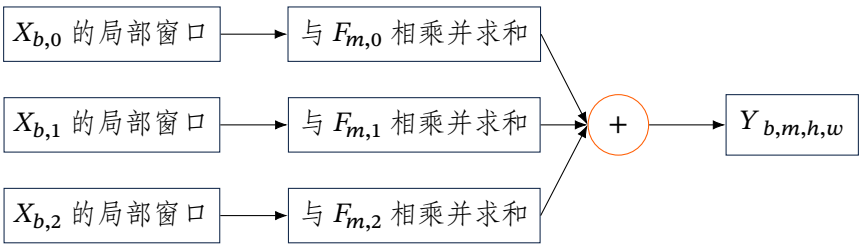


图 1 三个输入通道的贡献相加，生成固定输出通道中的一个空间位置。对每个新的输出通道，都需要另一组覆盖三个输入通道的卷积核。

1.4 卷积、池化与分类层的组合

卷积层产生包含空间位置信息的特征图，激活函数对其中每个标量执行非线性变换，池化（pooling）则把局部区域汇聚为较少的输出值。多次组合这些操作后，网络可以将较大感受野中的信息交给全连接层或其他分类结构。每个模块都接收和产生张量，上一层输出的通道数与空间尺寸直接决定下一层计算的输入形状。

经典卷积网络（LeNet-5）的一个形状序列是：32×32 单通道图像经过卷积得到 6×28×28，下采样为 $6\times14\times14$，再卷积为 $16\times10\times10$，再下采样为 $16\times5\times5$，之后形成 120 维、84 维的表示并产生 10 类输出。空间尺寸的收缩来自有效卷积和下采样，通道数的变化来自输出卷积核组数。较深的网络可以采用更多卷积层，也可以引入跨层连接或上采样分支；这些网络结构改变了张量之间的连接方式，卷积层内部仍可从同一套局部求和与通道组合规则出发实现。

卷积的局部连接主要减少每个输出所需的输入范围，参数共享主要减少独立权重数量，池化主要减少空间位置数。这三种机制作用于不同对象，因此需要分别分析参数量、计算量和中间张量存储量。

2 卷积与池化的前向计算

2.1 张量形状与求和维度

设一个批次（batch）包含 B 幅图像，每幅图像有 C 个输入通道，每个通道高 H、宽 W。卷积层产生 M 个输出通道，每个输出通道使用一组 C 个 $K \times K$ 的二维核。输入、权重和卷积输出分别记为

$$X \in \mathbb{R}^{B \times C \times H \times W}, \qquad F \in \mathbb{R}^{M \times C \times K \times K}, \qquad Y \in \mathbb{R}^{B \times M \times H_{\mathrm{o}} \times W_{\mathrm{o}}}. \tag{2}$$

宽度始终使用 W，权重始终使用 F，从而避免同一个字母同时表示两种对象。所有数组下标均从 0 开始。表2给出了各索引及其合法范围。

表 2 卷积前向计算的索引约定

索引	含义	范围
b	批次中的图像编号	$0 \le b < B$
c	输入通道编号	$0 \le c < C$
m	输出通道编号	$0 \le m < M$
h, w	输出的行、列坐标	$0 \le h < H_{\mathrm{o}}$, $0 \le w < W_{\mathrm{o}}$
p, q	核内的行、列偏移	$0 \le p < K$, $0 \le q < K$

先考虑步幅（stride）为 1、无填充（padding）、无空洞、每个输出通道组合全部输入通道的情形，并把偏置与激活留到后面单独处理。固定输出坐标 (b, m, h, w) 后，卷积计算为

$$Y_{b,m,h,w} = \sum_{c=0}^{C-1} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} F_{m,c,p,q} \, X_{b,c,h+p,w+q}. \tag{3}$$

这里的 h, w 确定窗口左上角；p, q 在窗口内部移动；c 遍历同一位置上的全部输入通道。一个输出值是长度为 CK^2 的内积。改变 m 时，输入窗口不变而权重组改变；改变 h, w 时，权重组不变而输入窗口移动；改变 b 时，处理另一幅图像而继续使用同一组参数。

式(3)在信号处理的严格命名中属于互相关（cross-correlation），因为核内的两个索引没有翻转。若采用翻转核的卷积约定，同一个窗口内应使用 $F_{m,c,K-1-p,K-1-q}$。CNN 的前向计算通常把式(3)直接称为卷积。实现时应保持公式中的索引顺序；核不对称时，额外翻转会改变输出。例如，核 $\left(\begin{smallmatrix}1 & 2\\ 3 & 4\end{smallmatrix}\right)$ 翻转后为 $\left(\begin{smallmatrix}4 & 3\\ 2 & 1\end{smallmatrix}\right)$，这两组权重与同一输入窗口的内积一般不同。

2.2 有效卷积的输出尺寸

有效卷积（valid convolution）只计算卷积核完全位于输入内部的位置。对于高度方向，窗口左上角从 $h = 0$ 开始，最下面一个核元素的行坐标是 $h+K-1$。要求该坐标不超过 $H-1$，就得到 $0 \le h \le H-K$，因此共有 $H-K+1$ 个合法起点。宽度方向同理，故

$$H_{\mathrm{o}} = H - K + 1, \qquad W_{\mathrm{o}} = W - K + 1, \qquad H \ge K,\ W \ge K. \tag{4}$$

这个计数包含起点 0 和终点 $H-K$，所以必须加 1。输出数组使用紧凑坐标：$Y_{b,m,0,0}$ 对应输入左上角的第一个完整窗口。若把输出响应画在核中心所对应的原图位置，两者的坐标原点不同，但它们代表相同的局部计算。

例如，7×7 输入与 5×5 核产生 3×3 输出；3×3 输入与 2×2 核产生 2×2 输出；86×86 输入与 7×7 核产生 80×80 输出。增加输出通道数只改变输出张量中的 M，不会改变这两个空间尺寸。步幅、填充或核内采样间隔发生变化时，需要重新按窗口合法起点计数，不能直接沿用式(4)。

对于高为 K_h、宽为 K_w 的矩形核，同样的计数分别给出 $H_{\mathrm{o}} = H - K_h + 1$ 和 $W_{\mathrm{o}} = W - K_w + 1$，每个输出的内积长度相应为 CK_hK_w。下面的推导与代码统一使用方形核，以一个 K 表示两个方向的大小。

2.3 步幅与零填充的尺寸关系

设矩形核的高、宽分别为 K_h, K_w，两个方向的步幅为正整数 S_h, S_w，输入上下各补 P_h 行零、左右各补 P_w 列零，其中填充量为非负整数。输出位置 (h, w) 的核内偏移 (p, q) 对应原输入坐标

$$i = hS_h - P_h + p, \qquad j = wS_w - P_w + q. \tag{5}$$

当 i, j 位于原输入范围内时读取 $X_{b,c,i,j}$，否则该项使用零。填充后的高度为 $H + 2P_h$，最后一个完整窗口的起点必须满足 $hS_h + K_h \le H + 2P_h$。在两个方向均至少容纳一个完整窗口时，输出尺寸为

$$H_{\mathrm{o}} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1, \qquad W_{\mathrm{o}} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1. \tag{6}$$

例如，$H = 6, W = 8$，使用 3×3 核、两个方向步幅均为 2，四侧各填充 1，输出为 3×4。左上输出的窗口从原输入坐标 $(-1, -1)$ 开始；右下输出的窗口从 $(3, 5)$ 开始。步幅限定了实际采样的起点，因此填充后末端仍可能有未被最后一个窗口覆盖的位置。

这些公式采用核内相邻元素连续采样的约定。以下算例、张量展开和代码继续使用步幅

1、无填充、无空洞的方形核；将它们推广到其他步幅或填充时，必须同时修改输出尺寸和输入坐标映射。

2.4 逐通道累加的完整算例

考虑一幅三通道图像，$C=3$、$H=W=4$，使用 $K=3$ 的核组产生一个输出通道，即 $M=1$。固定批次编号后，把三个输入通道简写为 X_0, X_1, X_2，相应核片简写为 F_0, F_1, F_2。同时固定唯一的输出通道 $m=0$，并简写 $Y_{h,w}=Y_{b,0,h,w}$。输入与核组为：

$$
\begin{aligned}
X_0 &= \begin{bmatrix} 1 & 2 & 0 & 1 \\ 1 & 1 & 3 & 2 \\ 0 & 2 & 2 & 0 \\ 2 & 1 & 0 & 3 \end{bmatrix}, \quad
X_1 = \begin{bmatrix} 0 & 2 & 1 & 0 \\ 0 & 3 & 2 & 1 \\ 1 & 1 & 0 & 2 \\ 2 & 1 & 0 & 3 \end{bmatrix}, \quad
X_2 = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 3 & 2 \\ 3 & 3 & 2 & 0 \\ 1 & 3 & 2 & 0 \end{bmatrix}, \\
F_0 &= \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 3 \\ 2 & 1 & 0 \end{bmatrix}, \qquad
F_1 = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 0 \\ 3 & 0 & 1 \end{bmatrix}, \qquad
F_2 = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 2 \\ 1 & 2 & 1 \end{bmatrix}.
\end{aligned}
\tag{7}
$$

输出高和宽均为 $4-3+1=2$。先求左上角的 $Y_{0,0}$，每个通道都取输入左上角的 3×3 窗口。按照窗口的行分别相乘再相加，可以直接看见每个通道如何贡献到同一个标量：

$$
\begin{aligned}
v_0 &= (1\cdot 1 + 2\cdot 1 + 0\cdot 1) + (1\cdot 2 + 1\cdot 2 + 3\cdot 3) + (0\cdot 2 + 2\cdot 1 + 2\cdot 0) \\
&= 3 + 13 + 2 = 18, \\
v_1 &= (0\cdot 1 + 2\cdot 2 + 1\cdot 3) + (0\cdot 1 + 3\cdot 1 + 2\cdot 0) + (1\cdot 3 + 1\cdot 0 + 0\cdot 1) \\
&= 7 + 3 + 3 = 13, \\
v_2 &= (1\cdot 0 + 2\cdot 1 + 1\cdot 1) + (0\cdot 1 + 1\cdot 0 + 3\cdot 2) + (3\cdot 1 + 3\cdot 2 + 2\cdot 1) \\
&= 3 + 6 + 11 = 20.
\end{aligned}
\tag{8}
$$

三个部分和合并后，$Y_{0,0}=18+13+20=51$。累加变量在开始计算这个输出前初始化一次为 0；处理完第一个通道后为 18，处理完第二个通道后为 31，处理完第三个通道后才写回最终值 51。如果在每个通道开始时清零并覆盖输出，就会丢失先前通道的贡献。

将窗口移到其余三个合法起点，三个通道各自贡献的 2×2 矩阵以及最终结果为

$$
Y = \begin{bmatrix} 18 & 23 \\ 20 & 16 \end{bmatrix} + \begin{bmatrix} 13 & 14 \\ 20 & 17 \end{bmatrix} + \begin{bmatrix} 20 & 13 \\ 20 & 15 \end{bmatrix} = \begin{bmatrix} 51 & 50 \\ 60 & 48 \end{bmatrix}. \tag{9}
$$

这四个输出使用相同的核组。它们的输入窗口彼此重叠，但输出地址不同；这种输入可复用、输出可独立写入的结构，正是后续并行实现的依据。

2.5 参数量、乘加次数与张量存储量

一次乘加（multiply-accumulate，MAC）表示将一个乘积累加到已有的和。式(3)中的每个输出需要 CK^2 次乘加，总输出元素数为 $BMH_\mathrm{o}W_\mathrm{o}$。如果把一次乘加按一次乘法和一次加法合计为两次浮点运算（floating-point operations，FLOPs），则

$$N_\mathrm{MAC} = BMH_\mathrm{o}W_\mathrm{o}CK^2, \qquad N_\mathrm{FLOP} = 2BMH_\mathrm{o}W_\mathrm{o}CK^2. \tag{10}$$

这里采用每次累加均计一次加法的惯例，与代码中从 0 开始执行乘加的形式对应。即使硬件把乘法和加法融合成一条指令，按照这个统计口径仍记为两次浮点运算。偏置、激活和池化需要另行计数。

参数存储与这些运算量具有不同的缩放关系。权重只有 MCK^2 个，与批次大小和输出空间位置数无关；输入有 $BCHW$ 个元素，输出有 $BMH_\mathrm{o}W_\mathrm{o}$ 个元素。使用 32 位浮点数时，每个数组的字节数等于元素数乘 4。增加 B 会增加输入、输出和卷积总运算量，而不会增加模型权重数量；增加图像尺寸会增加输出位置数，却仍复用同一组核参数。

在上面的三通道算例中，只有 $1\times3\times3^2 = 27$ 个权重，却需要为四个输出执行 $4\times27 = 108$ 次乘加，即 216 次浮点运算。若处理一百幅同样尺寸的图像，权重仍为 27 个，卷积运算量则增为 21 600 次浮点运算。

2.6 局部池化与非线性变换

池化把一个局部窗口内的多个数值汇总为一个输出。窗口沿空间轴移动，输入通道之间保持分离，因此常规空间池化保留批次大小和通道数量，同时改变每个通道的空间尺寸。常见汇总函数包括平均值、最大值、L^2 范数和加权平均。

设一个窗口包含 N^2 个值，按任意固定顺序记为 $z_1, \dots, z_{N^2}$。平均池化输出 $N^{-2}\sum_i z_i$，最大池化输出 $\max_i z_i$，L^2 池化输出 $(\sum_i z_i^2)^{1/2}$；带非负权重 a_i 且 $\sum_i a_i = 1$ 的加权平均输出 $\sum_i a_i z_i$。这些选择保留的局部信息不同：平均值反映整体响应，最大值保留最强响应，L^2 范数反映窗口中响应的合成幅度。若将平方和先除以 N^2 再开平方，得到的是均方根形式，需要与未归一化的 L^2 范数区分。

对卷积输出 Y 使用 $N\times N$ 窗口、步幅为 N、无填充的不重叠平均池化，空间输出尺寸为 $H_\mathrm{s} = \lfloor H_\mathrm{o}/N \rfloor$、$W_\mathrm{s} = \lfloor W_\mathrm{o}/N \rfloor$。池化输出坐标 (x, y) 所需窗口的左上角是 (Nx, Ny)，因此窗口

内元素为 $Y_{b,m,Nx+p,Ny+q}$，其中 $0 \le p,q < N$。若尺寸无法整除 N，最右侧或最下侧不足一个完整窗口的元素不参与这一约定下的池化。

进一步对平均值加上输出通道偏置 β_m，再施加 S 形激活函数（sigmoid），得到

$$S_{b,m,x,y} = \sigma\left(\frac{1}{N^2}\sum_{p=0}^{N-1}\sum_{q=0}^{N-1} Y_{b,m,Nx+p,Ny+q} + \beta_m\right), \qquad \sigma(z) = \frac{1}{1+\exp(-z)}, \qquad S \in \mathbb{R}^{B\times M\times H_s\times W_s}. \tag{11}$$

偏置 β_m 在所有图像和同一输出通道的全部空间位置上共享，所以这个偏置向量只包含 M 个数。平均池化本身没有需要学习的权重；附加偏置以后，这个组合模块具有 M 个可学习参数。

式(11)的运算顺序是先求平均、再加偏置、最后激活。非线性函数通常不能与平均运算交换。例如，窗口值为 $\{0,0,2,2\}$ 且偏置为 0 时，先平均再激活得到 $\sigma(1) \approx 0.7311$；先激活再平均则得到 $[\sigma(0)+\sigma(2)]/2 \approx 0.6904$。两者表示不同的层结构。固定通道偏置与平均可以交换，是因为 $\mathrm{avg}(Y+\beta_m) = \mathrm{avg}(Y) + \beta_m$；这项线性关系不能用来交换后面的非线性激活。

激活函数还可以使用整流线性单元（Rectified Linear Unit，ReLU），定义为 $\phi(z) = \max(0,z)$。它与 S 形函数都逐元素作用，不改变张量形状。具体实现应按照网络给定的函数和顺序执行，不能因为两个模块都称为激活层就互相替换。

2.7 池化的数值解释与独立性

对式(9)中的整个 2×2 输出做一次平均池化，得到 $(51+50+60+48)/4 = 52.25$。若偏置取 -52，则激活输入为 0.25，最终输出为 $\sigma(0.25) \approx 0.5622$。若只做最大池化，输出为 60；若做未归一化的 L^2 池化，输出为 $\sqrt{51^2+50^2+60^2+48^2} = \sqrt{11\,005}$。同一个窗口的不同汇总规则会产生不同结果，不能只根据输出尺寸识别池化操作。

池化可以降低对局部位置变化的敏感性，但这种效果取决于窗口边界与汇总方式。以一维最大池化作为简化例子，窗口宽度和步幅均为 2：输入 $[9,0,0,0]$ 和 $[0,9,0,0]$ 都得到 $[9,0]$；若响应继续右移一个位置，输入变为 $[0,0,9,0]$，输出就变为 $[0,9]$。因此，在一个窗口内部移动的特征可能给出相同结果，跨过窗口边界后则可能改变输出。空间下采样本身也不保证尺度不变性。

从并行计算角度看，(b,m,x,y) 不同的池化输出可以独立计算。一个线程可以读取所需的 N^2 个值，在寄存器中累加，除以 N^2，再执行偏置与激活，最后写回一个输出。这个方案与卷积中的单输出线程具有相同的组织原则：输出由线程唯一负责，局部窗口内的求和由该

线程完成。

卷积与池化之间还有一项可利用的数据复用。如果池化所需的卷积结果已由同一线程块产生，并且在写出前仍位于寄存器或共享内存中，就可以直接完成局部汇总，减少中间张量的全局写回和随后重读。合并实现时，必须保留式(11)的准确顺序，并确保每个池化窗口所需的全部卷积结果已经可用。窗口跨越线程块边界时，块内同步不能使另一个块的中间结果自动变得可用，需要重新安排分工或保留独立执行阶段。

3 直接卷积的并行实现与数据复用

3.1 张量地址与顺序参考实现

数学上的四维下标需要转换为实际内存地址。假设数组连续存储，最右侧的宽度索引变化最快，其次为高度、通道和批次，这就是这里采用的行主序（row-major order）布局。输入中的一个通道占 HW 个元素，一幅图像占 CHW 个元素，因此输入元素 $X_{b,c,i,j}$ 的线性偏移为 $bCHW + cHW + iW + j$。权重和输出使用同样的逐维展开规则：

$$\begin{aligned} \mathrm{off}_X(b,c,i,j) &= ((bC + c)H + i)W + j, \\ \mathrm{off}_F(m,c,p,q) &= ((mC + c)K + p)K + q, \\ \mathrm{off}_Y(b,m,h,w) &= ((bM + m)H_{\mathrm{o}} + h)W_{\mathrm{o}} + w. \end{aligned} \tag{12}$$

例如，输入宽度为 W 时，同一行中相邻像素的地址相差 1，同一列中相邻两行的地址相差 W。这既是正确索引的依据，也决定相邻线程的访存是否连续。

对于类型为 `float*` 的指针，`X[b,c,h,w]` 在 C++ 中包含逗号表达式，不能实现四维数组访问。代码需要使用真正的线性偏移。下面的辅助函数把四个索引映射到一个偏移，后三个参数依次给出后三维的长度。先把第一个索引转换为 `std::size_t`，可使后续的地址乘法使用适合数组大小的无符号整数类型。

代码 1 四维数组的行主序地址计算

```cpp
#pragma once
#include <cstddef>

#ifdef __CUDACC__
#define LEC11_HD __host__ __device__
#else
#define LEC11_HD
#endif

LEC11_HD inline std::size_t off4(
    int i0, int i1, int i2, int i3,
    int d1, int d2, int d3) {
  return ((static_cast<std::size_t>(i0) * d1 + i1)
          * d2 + i2) * d3 + i3;
}

#undef LEC11_HD
```

辅助函数在普通 C++ 中是主机函数，在 CUDA 编译中同时提供主机端和设备端版本。其调用约定完全相同：例如 `off4(b,c,i,j,C,H,W)` 返回输入偏移，而 `off4(m,c,p,q,C,K,K)` 返回权重偏移。

顺序卷积直接对应式(3)：外层四个循环枚举输出，内层三个循环完成一个输出的求和。以下函数假设 B, M, C, K 均为正数，$H, W \geq K$，三个数组已按式(2)分配，且输出数组不与输入或权重数组重叠。它只执行卷积，不在其中混入偏置或激活。

代码 2 逐输出累加的顺序卷积

```
#include "tensor_index.h"

void conv_forward_cpu(
    int B, int M, int C, int H, int W, int K,
    const float* X, const float* filter, float* Y) {
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  for (int b = 0; b < B; ++b)
    for (int m = 0; m < M; ++m)
      for (int h = 0; h < Hout; ++h)
        for (int w = 0; w < Wout; ++w) {
          float acc = 0.0f;
          for (int c = 0; c < C; ++c)
            for (int p = 0; p < K; ++p)
              for (int q = 0; q < K; ++q) {
                const auto xi = off4(b, c, h+p, w+q, C, H, W);
                const auto fi = off4(m, c, p, q, C, K, K);
                acc += X[xi] * filter[fi];
              }
          Y[off4(b, m, h, w, M, Hout, Wout)] = acc;
        }
}
```

每次进入新的 (b,m,h,w) 时创建并清零一个累加变量；全部输入通道和核位置处理完之后，写回一次输出。这个变量的生命周期明确了计算的归约边界：同一个输出的所有乘积必须合并在一起。连续处理不同输出时，重复读取的输入和权重来自运算自身的数据共享关系。

平均池化的顺序实现把每个输出窗口的起点改为 (Nx,Ny)，求和后除以窗口面积。其输入参数 `Hout` 和 `Wout` 表示前一卷积层的输出尺寸，`Hs` 和 `Ws` 才是当前池化结果的尺寸。假设 $N>0$ 且至少存在一个完整池化窗口，整数除法直接给出前述向下取整。

代码 3 平均池化、通道偏置与 S 形激活

```cpp
#include <cmath>
#include "tensor_index.h"

static float sigmoid_cpu(float z) {
  const float e = std::exp(-std::fabs(z));
  return z >= 0.0f ? 1.0f / (1.0f + e) : e / (1.0f + e);
}

void pool_forward_cpu(
    int B, int M, int Hout, int Wout, int N,
    const float* Y, const float* bias, float* S) {
  const int Hs = Hout / N;
  const int Ws = Wout / N;
  for (int b = 0; b < B; ++b)
    for (int m = 0; m < M; ++m)
      for (int x = 0; x < Hs; ++x)
        for (int y = 0; y < Ws; ++y) {
          float acc = 0.0f;
          for (int p = 0; p < N; ++p)
            for (int q = 0; q < N; ++q)
              acc += Y[off4(b, m, N*x+p, N*y+q,
                            M, Hout, Wout)];
          const float mean = acc / (static_cast<float>(N) * N);
          S[off4(b, m, x, y, M, Hs, Ws)] =
              sigmoid_cpu(mean + bias[m]);
        }
}
```

激活函数使用了与 $1/(1+\exp(-z))$ 等价的分支形式。$z \ge 0$ 时计算 $1/(1+\exp(-z))$，$z < 0$ 时计算 $\exp(z)/(1+\exp(z))$；两种分支中指数的自变量都不大于 0，从而避免为一个最终接近 0 的激活值先计算很大的正指数。

3.2 输出并行与归约维度

卷积中的并行性可以直接从输出下标和求和下标区分。批次索引 b、输出通道索引 m、输出坐标 h, w 共同确定一个唯一的输出地址。不同四元组之间没有结果依赖，因此可以并行执行。只要每个四元组分配给唯一线程，各线程就能独立读输入、读权重并写自己的输出。

输入通道 c 和核内坐标 p, q 则属于同一个输出的归约维度（reduction dimension）。它们对应许多需要相加的部分乘积。基础方案让一个线程顺序完成全部 CK^2 项，因而不需要原子操作；如果把这些求和项分给不同线程，就需要归约或原子累加来合并部分结果，并处理累加初值与最终写回的时序。

这一区分也解释了为什么只把不同输出通道交给不同线程可能并行度不足：M 有时只

有几个，而全部输出位置数为 $BMH_{\mathrm{o}}W_{\mathrm{o}}$。利用空间位置和批次中的独立图像，可以同时暴露更多任务。较深层的特征图可能更小，但通道数和单个输出的累加长度可能更大；线程粒度与共享数据的组织因此需要结合具体形状选择。

3.3 空间分块与线程索引

令每个线程块计算固定图像、固定输出通道中的一个 $T \times T$ 输出分块。块内配置 $T \times T$ 个线程，线程的横坐标对应输出列，纵坐标对应输出行。对于一张大小为 $H_{\mathrm{o}} \times W_{\mathrm{o}}$ 的特征图，横向分块数为 $n_w = \lceil W_{\mathrm{o}}/T \rceil$，纵向分块数为 $n_h = \lceil H_{\mathrm{o}}/T \rceil$。向上取整保证最右侧和最下侧剩余的位置也有对应线程。

将空间分块按行展开，记其编号为 t，则 $0 \le t < n_h n_w$。分块所在的行、列分别为 $\lfloor t/n_w \rfloor$ 与 $t \bmod n_w$，所以该块左上角的输出坐标为 $(T\lfloor t/n_w \rfloor, T(t \bmod n_w))$。使用以下网格维度，就能把输出通道、空间分块和图像批次分别放在三个网格轴上：

$$
\begin{aligned}
&\text{线程块维度} = (T, T, 1), && \text{网格维度} = (M, n_h n_w, B),\\
&m = \texttt{blockIdx.x}, && b = \texttt{blockIdx.z},\\
&h = T\left\lfloor \frac{\texttt{blockIdx.y}}{n_w} \right\rfloor + \texttt{threadIdx.y},\\
&w = T(\texttt{blockIdx.y} \bmod n_w) + \texttt{threadIdx.x}.
\end{aligned}
\tag{13}
$$

这里 CUDA 网格的 x 维代表输出通道，图像水平方向的分块数却是 n_w。图像的空间方向与网格轴的名称不必相同，关键是恢复坐标时使用一致的对应关系。

例如，$M = 4$、$H_{\mathrm{o}} = W_{\mathrm{o}} = 8$、$T = 4$ 时，每个输出通道需要 $2 \times 2 = 4$ 个空间分块，单幅图像的网格大小为 $(4, 4, 1)$，共有 16 个线程块。空间分块 $t = 0, 1$ 分别位于上排的左、右两侧，$t = 2, 3$ 分别位于下排的左、右两侧。固定 $m = 2$、$t = 3$，再取块内坐标 $\texttt{threadIdx.x} = 1$、$\texttt{threadIdx.y} = 2$，就得到 $h = 6, w = 5$，因此该线程负责 $Y_{0,2,6,5}$。

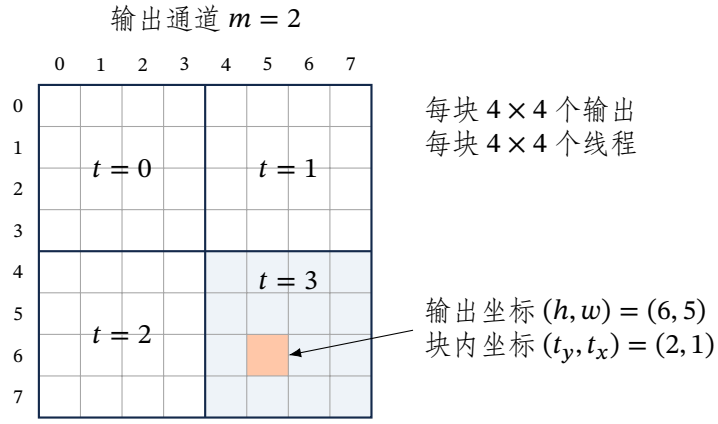


图 2 一张 8×8 特征图的空间分块。浅蓝区域由分块 $t = 3$ 负责，橙色单元由该块中纵坐标为 2、横坐标为 1 的线程计算。

3.4 基础卷积核与边界处理

基础设备函数用式(13)替代顺序程序的四个输出循环，其余的 c, p, q 三重累加保持不变。模板参数 T 是编译时确定的分块边长，启动时的线程块必须与它一致；`tiles_w` 对应上面的 n_w。输入、权重和输出指针都指向设备内存。

代码 4 每个线程计算一个卷积输出的设备函数

```cpp
#include <cuda_runtime.h>
#include "tensor_index.h"

template<int T>
__global__ void conv_forward_kernel(
    int M, int C, int H, int W, int K, int tiles_w,
    const float* X, const float* filter, float* Y) {
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  const int b = blockIdx.z;
  const int m = blockIdx.x;
  const int h = (blockIdx.y / tiles_w) * T + threadIdx.y;
  const int w = (blockIdx.y % tiles_w) * T + threadIdx.x;
  if (h >= Hout || w >= Wout) return;

  float acc = 0.0f;
  for (int c = 0; c < C; ++c)
    for (int p = 0; p < K; ++p)
      for (int q = 0; q < K; ++q) {
        const auto xi = off4(b, c, h+p, w+q, C, H, W);
        const auto fi = off4(m, c, p, q, C, K, K);
        acc += X[xi] * filter[fi];
      }
  Y[off4(b, m, h, w, M, Hout, Wout)] = acc;
}
```

使用向上取整后，一些线程会落在输出范围之外。对于这个没有块内同步的基础核，可以直接让这些线程返回。对剩余线程，有 $h \le H-K$、$p \le K-1$，从而 $h+p \le H-1$；宽度方向同理。这说明，只要输出坐标合法，整个有效卷积窗口中的输入地址就合法，无须在内层循环中再对每个输入元素重复判断。

相应主机函数计算网格并启动设备函数。下面取 $T=16$，每块为 256 个线程。数组应提前分配，且网格维度与数组大小需要位于所用设备和索引类型的支持范围内。

代码 5 包含批次维度与边缘分块的卷积启动函数

```
cudaError_t launch_conv_forward(
    int B, int M, int C, int H, int W, int K,
    const float* dX, const float* dFilter, float* dY) {
  if (B <= 0 || M <= 0 || C <= 0 || K <= 0 || H < K || W < K)
    return cudaErrorInvalidValue;
  constexpr int T = 16;
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  const int tiles_w = (Wout + T - 1) / T;
  const int tiles_h = (Hout + T - 1) / T;
  const dim3 block(T, T, 1);
  const dim3 grid(M, tiles_h * tiles_w, B);
  conv_forward_kernel<T><<<grid, block>>>(
      M, C, H, W, K, tiles_w, dX, dFilter, dY);
  return cudaGetLastError();
}
```

启动函数返回的是内核启动阶段的错误状态。内核执行通常相对于主机异步进行；主机读取计算结果前，还需要在执行流程中完成相应的同步或阻塞式结果复制。这个时序安排与卷积的数值公式分开处理。

输出尺寸为 34×34、$T=16$ 时，必须启动 3×3 个空间分块。若直接用整数除法计算 $34/16=2$，就只覆盖前 32 行和前 32 列。向上取整加输出边界检查使同一个核能够同时处理可整除和不可整除的形状。

平均池化也可以沿用同一输出映射，把 $H_{\mathrm{o}}, W_{\mathrm{o}}$ 换成 $H_{\mathrm{s}}, W_{\mathrm{s}}$，让每个线程负责一个 $S_{b,m,x,y}$，再按式(11)访问 $(Nx+p, Ny+q)$。这时改变的是线程内部的输入地址与汇总运算，批次、通道和空间分块的组织方式仍然适用。

基础核中的输入全部从全局内存读取。把输出分给二维线程块，已经解决了并行任务分配；要进一步让同一块中的线程复用输入数据，还需要显式引入共享内存。

3.5 共享内存中的输入复用

相邻输出的输入窗口大量重叠。对于一个完整的 $T \times T$ 输出分块，固定一个输入通道，如果每个线程都从全局内存独立读取自己的 $K \times K$ 窗口，共发出 T^2K^2 次输入读取请求。其中，同一个输入元素可能被块内多个线程重复请求。

求这些窗口的联合覆盖范围,就能确定需要协作加载的数据。设输出块左上角为 (h_0, w_0)，一个线程需要的输入行是 $h_0 + t_y + p$，其中 $0 \le t_y < T$、$0 \le p < K$。它的最小值为 h_0，最大值为 $h_0 + T + K - 2$，因此联合输入区域在高度方向包含 $T + K - 1$ 个位置。宽度方向相同，所以输入分块大小为 $(T + K - 1) \times (T + K - 1)$。

输出分块之外、为完成边缘输出而额外加载的输入区域称为晕区 (halo)。将整个联合输入区域加载到共享内存，每个输入元素在这个块内只从全局内存取一次，后面的所有局部乘加都可读取共享缓冲区。对于单个输入通道，完整分块的读取请求减少倍数为

$$R_{\text{load}} = \frac{\text{独立读取各窗口的次数}}{\text{协作加载联合输入区域的次数}} = \frac{T^2K^2}{(T + K - 1)^2}. \tag{14}$$

例如，$T = 16, K = 7$ 时，独立读取需要 $256 \times 49 = 12\,544$ 次输入请求；联合输入块只有 $22^2 = 484$ 个元素，于是 $R_{\text{load}} = 12\,544/484 \approx 25.92$。若逐个处理 C 个输入通道，分子和分母都乘 C，该比值保持不变。

这个比值刻画指定条件下请求的全局输入读取数。共享内存加载、同步、乘加和输出写回仍然需要时间，缓存也可能已服务了基础实现的一部分重复读取，因此不能把 25.92 直接当作运行时间的加速倍数。相邻线程块的晕区仍可能重叠，同一输入区域也可能由不同输出通道对应的线程块重复读取；式(14)只计算一个块内部的协作收益。

3.6 协作加载、同步与权重复用

输入分块有 $(T + K - 1)^2$ 个元素，往往多于块内的 T^2 个线程，因此每个线程只装入一个元素并不足够。令 $Q = T + K - 1$，块内线性线程号为 $u = t_yT + t_x$。让线程 u 负责所有满足 $a = u + jT^2 < Q^2$ 的输入线性位置，其中 $j = 0, 1, 2, \ldots$。每个合法 a 恰好具有一个模 T^2 的余数，因此所有线程的加载范围不重叠，并且覆盖整个输入分块。由 a 恢复共享缓冲区坐标为 $r = \lfloor a/Q \rfloor$、$s = a \bmod Q$，对应的原输入位置是 $(h_0 + r, w_0 + s)$。

使用同一共享缓冲区依次处理不同输入通道时，每个通道需要遵守三个阶段的时序：

1. 所有线程协作把该通道所需的 $Q \times Q$ 输入区域放入共享内存。边缘块中，超出原输入的加载位置填入 0；合法输出所需要的输入仍全部来自有效窗口。

2. 全块执行同步。随后，负责合法输出的线程读取共享缓冲区中的 $K \times K$ 窗口，把本通道贡献累加到各自的寄存器中。

3. 在下一通道覆盖这块共享缓冲区之前再次全块同步，确保当前通道的数据已经被所有需要它的线程读完。

第一道同步防止读取尚未加载完的数据，第二道同步防止较快线程过早覆盖较慢线程仍在使用的数据。输出范围外的线程虽然不产生结果，仍可能承担晕区加载并参与同步。因此，代码4中的越界提前返回只适用于当前没有块内同步的基础核；引入共享缓冲区以后，应保留全块加载与同步，只对累加和最终输出写回设置有效性条件。

按通道复用缓冲区时，共享内存大小约为 $4Q^2$ 字节，而不是 $4CQ^2$ 字节；累加变量跨通道保存在每个线程的寄存器中。一次装入所有通道也能实现相同计算，但共享内存占用随 C 增长。如何选择取决于可用容量、同步成本以及一个计算单元上能够同时驻留多少个线程块。

权重还有另一种复用方式。在固定输出通道的基础映射中，同一线程束（warp）内的线程处理不同空间位置，却在相同的 c, p, q 循环迭代读取相同的 $F_{m,c,p,q}$。当核组能够放入常量内存（constant memory）且访问保持这种同址特征时，可以利用广播让多个线程共享一次权重取值。若另一种线程映射让同一线程束同时处理不同输出通道，权重地址就可能不同，需要重新分析访问行为。代码4使用普通权重指针，常量内存属于可在该基础上采用的存储优化。

在图像输入方面，固定的 c, p, q 下，相邻横向线程读取相邻列，行主序布局因而有利于连续访问。不同输出通道的线程块还会访问同一幅图像中的相同区域；这些块有可能利用缓存中的输入，但实际复用取决于调度和缓存容量。若希望在一次协作计算中显式共享跨输出通道的输入，可以把多个输出通道一起纳入分块，或者利用下面的矩阵乘法表示。

4 卷积的矩阵表示与输入展开

4.1 从局部内积到矩阵乘法

固定一幅图像和一个输出位置 (h, w)，把该位置所需的全部输入通道窗口连接成一个长度为 CK^2 的向量。每个输出通道都将自己的核组与这个向量做内积，因此可以把不同输出通道的权重排成矩阵的各行，把不同输出空间位置所需的输入向量排成另一个矩阵的各列。矩阵乘法中的一个行列内积就生成一个卷积输出。

输入展开（image-to-column，im2col）是按照上述方式重新组织输入窗口的操作。令 $D = CK^2$ 为一个输出所需内积的长度，$L = H_{\mathrm{o}} W_{\mathrm{o}}$ 为每个输出通道的位置数。对固定图像 b，定义核矩阵 $A \in \mathbb{R}^{M \times D}$、展开矩阵 $U_b \in \mathbb{R}^{D \times L}$，以及矩阵形式的输出 $V_b \in \mathbb{R}^{M \times L}$。

首先固定展开顺序。核内列坐标 q 变化最快，其次为核内行坐标 p，最后为输入通道 c，所以展开行号为 $r = cK^2 + pK + q$。输出列坐标 w 变化最快，其次为输出行坐标 h，所以展开列号为 $\ell = hW_{\mathrm{o}} + w$。由这两个编号得到

$$A_{m,r} = F_{m,c,p,q}, \qquad (U_b)_{r,\ell} = X_{b,c,h+p,w+q}, \qquad V_b = AU_b. \tag{15}$$

展开矩阵的一列包含一个输出位置所需的全部 C 个窗口，列内每连续 K^2 行属于一个输入通道。展开矩阵的一行则固定了一个输入通道及一个核内偏移，让输出窗口的左上角遍历全部空间位置。

矩阵乘法与卷积的等价性可以逐元素验证。对 $\ell = hW_{\mathrm{o}} + w$，用 r 与 (c, p, q) 的一一对应关系重写内积，得到

$$\begin{aligned} (V_b)_{m,\ell} &= \sum_{r=0}^{D-1} A_{m,r} (U_b)_{r,\ell} \\ &= \sum_{c=0}^{C-1} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} F_{m,c,p,q} X_{b,c,h+p,w+q} = Y_{b,m,h,w}. \end{aligned} \tag{16}$$

恢复卷积输出时，只需把 V_b 的每一行按 $H_{\mathrm{o}} \times W_{\mathrm{o}}$ 解释。若 V_b 按行连续存储，其内部顺序与对应图像的 $Y_{b,:,:,:}$ 已经一致，这一步不需要再搬运元素。

这套组织方式使同一展开列可被所有 M 个输出通道使用，也使同一权重行可用于全部 L 个空间位置。随后可以用分块矩阵乘法，把这两种复用显式放入共享内存和寄存器。

4.2 三通道、双输出的完整矩阵算例

取 $B=1$、$C=3$、$M=2$、$H=W=3$、$K=2$。输出尺寸为 $H_{\mathrm{o}}=W_{\mathrm{o}}=2$，因而 $D=12$、$L=4$。固定唯一图像后，三个输入通道为

$$X_0=\begin{bmatrix}1&2&0\\1&1&3\\0&2&2\end{bmatrix},\qquad X_1=\begin{bmatrix}0&2&1\\0&3&2\\1&1&0\end{bmatrix},\qquad X_2=\begin{bmatrix}1&2&1\\0&1&3\\3&3&2\end{bmatrix}. \tag{17}$$

将两组卷积权重写成下面的 2×12 矩阵。每一行对应一个输出通道，每连续四列对应一个输入通道上的 2×2 核，竖线仅用于标出通道之间的分界：

$$A=\left[\begin{array}{cccc|cccc|cccc}1&1&2&2&1&1&1&1&0&1&1&0\\1&0&0&1&2&1&2&1&1&2&2&0\end{array}\right]. \tag{18}$$

例如，第一行的前四个数 $[1,1,2,2]$ 对应 $F_{0,0}=\left(\begin{smallmatrix}1&1\\2&2\end{smallmatrix}\right)$，下一组 $[1,1,1,1]$ 对应 $F_{0,1}$。把每组连续四个数按两行两列恢复，就能得到全部六个核片。

四个输出位置按照 $(0,0),(0,1),(1,0),(1,1)$ 排列为四列。依次取各位置上的输入窗口，并按通道、核行、核列顺序展开，得到

$$U_0=\left[\begin{array}{cccc}1&2&1&1\\2&0&1&3\\1&1&0&2\\1&3&2&2\\\hline0&2&0&3\\2&1&3&2\\0&3&1&1\\3&2&1&0\\\hline1&2&0&1\\2&1&1&3\\0&1&3&3\\1&3&3&2\end{array}\right]. \tag{19}$$

上、中、下三个四行区域分别来自输入通道 $0,1,2$。例如，第三列对应 $(h,w)=(1,0)$，其最后四个数必须来自 X_2 中的窗口 $\left(\begin{smallmatrix}0&1\\3&3\end{smallmatrix}\right)$，所以这一段为 $[0,1,3,3]^{\mathrm{T}}$。核组和输入窗口必须按相同的 (c,p,q) 顺序展开；只改变一侧的排列通常会改变内积。

第一个输出通道的左上值由 A 第一行与 U_0 第一列相乘得到。把十二项按三个输入通道分组，计算为

$$\begin{aligned}(V_0)_{0,0} &= (1\cdot 1 + 1\cdot 2 + 2\cdot 1 + 2\cdot 1)\\ &\quad + (1\cdot 0 + 1\cdot 2 + 1\cdot 0 + 1\cdot 3)\\ &\quad + (0\cdot 1 + 1\cdot 2 + 1\cdot 0 + 0\cdot 1) = 7 + 5 + 2 = 14.\end{aligned} \tag{20}$$

对两行和四列完成全部内积后，得到

$$V_0 = AU_0 = \begin{bmatrix} 14 & 20 & 15 & 24 \\ 12 & 24 & 17 & 26 \end{bmatrix}, \qquad Y_{0,0} = \begin{bmatrix} 14 & 20 \\ 15 & 24 \end{bmatrix}, \quad Y_{0,1} = \begin{bmatrix} 12 & 24 \\ 17 & 26 \end{bmatrix}. \tag{21}$$

这里 $Y_{0,0}$ 与 $Y_{0,1}$ 分别固定图像编号和输出通道编号，保留下来的矩阵行、列对应输出的两个空间坐标。矩阵乘法只是重新排列了乘积项的组织方式，输出数值与逐通道卷积一致。

4.3 展开索引与实际数组实现

权重张量的线性偏移可以写成 $mD+r$，与 $M\times D$ 行主序矩阵的偏移相同，因此在既定布局下，核矩阵 A 可以直接使用原来的权重数组。输入窗口存在重叠，展开矩阵则需要把每个窗口所需的值放入指定行列。显式存放全部展开矩阵时，数组形状为 $B\times D\times L$，元素 $(U_b)_{r,\ell}$ 的偏移为 $(bD+r)L+\ell$。

顺序展开代码按图像、输入通道和核内偏移确定矩阵行，再让输出位置遍历矩阵列。两个输出位置即使需要相同的原输入元素，也分别写入自己的展开位置。

代码 6 按批次保存展开矩阵的顺序实现

```cpp
#include "tensor_index.h"

void unroll_cpu(
    int B, int C, int H, int W, int K,
    const float* X, float* U) {
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  const std::size_t D = static_cast<std::size_t>(C) * K * K;
  const std::size_t L = static_cast<std::size_t>(Hout) * Wout;
  for (int b = 0; b < B; ++b)
    for (int c = 0; c < C; ++c)
      for (int p = 0; p < K; ++p)
        for (int q = 0; q < K; ++q) {
          const std::size_t r =
              (static_cast<std::size_t>(c) * K + p) * K + q;
          for (int h = 0; h < Hout; ++h)
            for (int w = 0; w < Wout; ++w) {
              const auto ell = static_cast<std::size_t>(h) * Wout + w;
              U[(static_cast<std::size_t>(b) * D + r) * L + ell] =
                  X[off4(b, c, h+p, w+q, C, H, W)];
            }
        }
}
```

另一种组织方式是直接枚举展开矩阵的位置，再反求应当读取的原输入。给定 r 和 ℓ，由于 r 先按通道分成长度为 K^2 的区段，再在每段中按行展开，所以逆映射为

$$\begin{aligned} c &= \lfloor r/K^2 \rfloor, \quad p = \lfloor (r \bmod K^2)/K \rfloor, \qquad q = r \bmod K, \\ h &= \lfloor \ell/W_{\mathrm{o}} \rfloor, \quad w = \ell \bmod W_{\mathrm{o}}, \qquad (U_b)_{r,\ell} = X_{b,c,h+p,w+q}. \end{aligned} \tag{22}$$

例如，$C=3, K=2$ 时，$r=9$ 对应 $(c,p,q)=(2,0,1)$；若 $W_{\mathrm{o}}=2$ 且 $\ell=2$，则 $(h,w)=(1,0)$，因此展开位置 $(9,2)$ 读取第三输入通道中的 $(1,1)$。

设备实现可以使用一维网格，把完整 $B \times D \times L$ 数组的线性位置分给各线程。下面的网格步进循环（grid-stride loop）让每个线程在处理完自己的起始位置后，跨过整个网格的线程总数继续处理。所有合法位置都被恰好一个线程负责，即使数组大于一次启动的线程总数，也能够覆盖全部元素。

代码 7 从展开位置反求输入地址的设备实现

```cuda
#include <cuda_runtime.h>
#include "tensor_index.h"

__global__ void unroll_kernel(
    int B, int C, int H, int W, int K,
    const float* X, float* U) {
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  const std::size_t D = static_cast<std::size_t>(C) * K * K;
  const std::size_t L = static_cast<std::size_t>(Hout) * Wout;
  const std::size_t total = static_cast<std::size_t>(B) * D * L;
  const std::size_t step =
      static_cast<std::size_t>(blockDim.x) * gridDim.x;

  for (std::size_t idx =
           static_cast<std::size_t>(blockIdx.x) * blockDim.x +
   threadIdx.x;
       idx < total; idx += step) {
    const std::size_t ell = idx % L;
    const std::size_t r = (idx / L) % D;
    const int b = idx / (D * L);
    const int c = r / K / K;
    const int p = (r / K) % K;
    const int q = r % K;
    const int h = ell / Wout;
    const int w = ell % Wout;
    U[idx] = X[off4(b, c, h+p, w+q, C, H, W)];
  }
}
```

该设备函数使用一维线程块和一维网格，块数及每块线程数均应为所用设备支持的正值。循环中的 `idx` 就是展开数组的目标偏移：先用余数恢复 ℓ，再恢复 r 与图像编号 b。同一批相邻线程写入连续展开元素；当它们仍位于同一展开行和同一输出空间行时，对原输入的读取也沿宽度方向连续。跨输出行、跨核偏移或跨输入通道时，输入地址会相应跳转。

展开操作的输出位置没有重叠写入，因而不需要原子操作。重复出现在不同窗口中的输入是重复读取的数据，它们进入展开矩阵中的不同地址。这种复制正是显式展开产生额外存储需求的原因。

4.4 批次组织与输出内存顺序

逐图计算 $V_b = AU_b$ 时，U 按 $B \times D \times L$ 存储，每个 V_b 写到该图像对应的 $M \times L$ 区域，自然得到 $B \times M \times L$ 的输出顺序。核矩阵 A 在整个批次中共享，不需要为每幅图像复制一份权重。批量矩阵乘法也可以使用相同的数学组织。

也可以把所有图像的展开列连接成 $U_{\text{all}} \in \mathbb{R}^{D \times (BL)}$，再一次计算 $AU_{\text{all}} \in \mathbb{R}^{M \times (BL)}$。具体地，$(U_{\text{all}})_{r,bL+\ell} = (U_b)_{r,\ell}$，其行主序偏移为 $r(BL) + bL + \ell$，与原数组的 $(bD + r)L + \ell$ 不同。因此，拼列需要按新顺序组织输入，不能仅改变原数组的形状解释。

在这种表示中，每个输出通道的一整行包含全部图像的位置，因此矩阵乘积的行主序布局对应 $M \times B \times L$。其地址与原来按批次优先存储的输出地址分别为

$$\text{off}_{\text{concat}} = m(BL) + bL + \ell, \qquad \text{off}_{\text{batch}} = (bM + m)L + \ell. \tag{23}$$

例如，$B = 2, M = 2, L = 4$ 时，图像 $b = 1$、输出通道 $m = 0$ 的第一个元素，在拼列乘积中的偏移为 4，在批次优先输出中的偏移为 8。因此，选择拼列计算时，展开数组本身就要采用对应布局，乘积也需要按后续层期望的顺序重排，或让后续计算显式接收这种新布局。矩阵形状相容只保证数学乘法成立，实际数组还必须使用相应的元素顺序。

4.5 展开存储与矩阵分块复用

显式展开把窗口重叠转化为数组中的重复元素。全部输入共有 $BCHW$ 个元素，全部展开矩阵共有 $BDL = BCK^2H_{\text{o}}W_{\text{o}}$ 个元素，因此展开数组相对于输入数组的大小比为

$$\rho_{\text{expand}} = \frac{BCK^2H_{\text{o}}W_{\text{o}}}{BCHW} = \frac{K^2(H - K + 1)(W - K + 1)}{HW}. \tag{24}$$

式(19)中的展开矩阵有 $12 \times 4 = 48$ 个元素，原输入有 $3 \times 3 \times 3 = 27$ 个元素，大小比为 $48/27 = 16/9 \approx 1.78$。当 H, W 远大于 K 时，$H_{\text{o}}W_{\text{o}}$ 接近 HW，该比值趋近 K^2。这不是每个输入元素都复制相同次数：内部元素可能属于许多窗口，边界元素属于较少窗口；式(24)比较的是整个数组的大小。

若每个元素占 s 字节，完整展开工作区需要 $sBDL$ 字节。代码6或7对每个展开位置读取一次原输入并写入一次展开数组，因而产生 BDL 次输入读取请求和 BDL 次展开写入；随后矩阵乘法还要读取展开数组。缓存、事务合并和矩阵分块会影响实际显存传输量，因此这里的元素请求计数不能直接等同于硬件上的总传输字节数。

展开并没有减少卷积的乘加次数。对每幅图像计算一个 $M \times D$ 与 $D \times L$ 的矩阵乘积，仍

需 MDL 次乘加，整个批次为 $BMDL$ 次。其优势在于把输入、权重和输出之间的复用关系组织成适合通用矩阵乘法（general matrix multiplication，GEMM）的规则形式。

考虑矩阵乘法中的一个输出块，它覆盖 T_m 个输出通道和 T_n 个展开列；一次沿求和维度处理 T_k 个元素。这里两个输出方向是通道 m 和线性化空间位置 ℓ，并非直接卷积空间块中的高度和宽度。在这一阶段，线程块将 $T_m \times T_k$ 的核矩阵块与 $T_k \times T_n$ 的展开矩阵块放入共享内存，再计算它们对输出块的贡献。对于三个方向均完整有效的分块，加载元素数与乘加数分别为

$$N_{\text{load}} = T_m T_k + T_k T_n, \qquad N_{\text{MAC}} = T_m T_n T_k. \tag{25}$$

每个核元素可用于 T_n 个输出位置，每个展开元素可用于 T_m 个输出通道。相比只让一个线程块负责单一输出通道的空间块，矩阵分块还能够在块内显式复用不同输出通道需要的同一输入窗口。完成全部求和块后，累加结果才写回对应的输出区域。

输出块更大通常带来更多块内复用，也需要更多共享内存、累加寄存器和线程工作量。边缘块中不足 T_m, T_n, T_k 的部分还会产生无效位置。选择块尺寸时，需要同时考虑有效工作比例和可驻留的线程块数量，不能只追求单个块的复用次数。

矩阵乘法的效率还取决于三个维度 M, D, L 各自的大小。一个维度很大，并不能补偿另一个维度中的大量无效分块位置。例如，第一层卷积的矩阵形状为 4×49 与 49×6400：空间位置很多，但输出通道只有 4 个。若沿输出通道方向固定使用 $T_m = 16$ 的分块，则该方向只有四分之一的位置有效；若求和分块宽度为 $T_k = 16$，最后一个求和块也只有一个有效元素。调整分块形状或让线程承担不同数量的输出，可以减少这些损失，但会改变资源占用和访存组织。较小的矩阵还可能使内核启动开销占据较大比例。因此，将卷积写成 GEMM 提供了规则的复用结构，实际性能仍需结合矩阵形状、边缘利用率和完整执行阶段衡量。

4.6 隐式展开与共享内存、寄存器联合分块

隐式展开（implicit unrolling）保留 $V_b = AU_b$ 的矩阵计算方式，但不在全局内存中存放完整 U_b。当分块矩阵乘法准备加载展开矩阵中的某个元素时，直接利用式(22)算出它在原输入中的地址，将原输入值送入当前共享内存块。展开矩阵只作为索引关系存在。

固定图像 b，设当前展开矩阵块的左上位置为 (r_0, ℓ_0)，块内位置为 (k, j)。首先得到 $r = r_0 + k$、$\ell = \ell_0 + j$；若 $r < D$ 且 $\ell < L$，就从 (r, ℓ) 恢复 c, p, q, h, w，再读取 $X_{b,c,h+p,w+q}$。核矩阵块中的元素仍是 A_{m_0+i, r_0+k}。两块就绪后，输出块的每个元素累加

$$\text{acc}_{i,j} \mathrel{+}= \sum_{k=0}^{T_k-1} A_{m_0+i, r_0+k} \, (U_b)_{r_0+k, \ell_0+j}. \tag{26}$$

超出 M, D, L 范围的分块位置填零，最终只写回合法输出。这里的零用于补齐矩阵边缘块，并未给原输入增加卷积填充；合法输出仍然使用原输入内部的完整窗口。共享内存块的加载、使用和下一轮覆盖之间，同样需要正确的块内同步。

把展开地址计算并入矩阵乘法，是核函数融合（kernel fusion）的一种形式。它省去了完整展开数组的全局写入和独立展开阶段的启动开销；原先读取 U 的动作改为按需读取 X，无需再从全局内存读取物化的展开数组，同时也省去了 *sBDL* 字节的完整展开工作区。共享内存中的局部块与输出累加寄存器仍然需要空间。原输入的一个元素可能因属于多个窗口或多个线程块而再次被读取，隐式展开不会自动消除所有重叠读取。它还增加了加载路径中的取整、取余和地址计算，因此是否更快取决于减少的存储开销与新增的寻址成本。

在共享内存分块之上，还可以让每个线程保存一个 $R_m \times R_n$ 的输出子块。对一个求和位置，该线程从共享内存取出 R_m 个权重和 R_n 个输入值，组成两组短向量；它们的所有两两乘积更新 $R_m R_n$ 个寄存器累加器。这样，一次取到的权重可在本线程内用于 R_n 个输出位置，一次取到的输入可用于 R_m 个输出通道。线程块之间的分工、共享内存块和线程内寄存器子块由此形成两级复用。

寄存器子块增大后，每个线程要保存更多累加器和临时值，可能减少同时驻留的线程，过高的寄存器需求还可能造成溢出存储。因此，共享内存与寄存器联合分块应平衡数据复用和执行资源占用。四种实现的主要区别可归纳为表3；联合分块既可用于显式展开后的矩阵乘法，也可用于隐式展开。

表 3 卷积实现方式与主要数据组织

实现方式	数据复用方式	主要代价或约束
直接空间分块	协作装入带晕区的输入；适合时广播权重	晕区及不同块的重复读取；通道间同步
显式展开后乘法	完整保存窗口列，再用矩阵分块共享输入和权重	展开工作区、展开读写与独立执行阶段
隐式展开乘法	装入矩阵块时从原输入取值	地址计算；输入访问仍可能重复
共享内存与寄存器联合分块	块内共享数据，线程内复用短向量	累加寄存器与共享内存占用

这些实现计算的是同一组实数乘积与求和，但浮点加法不满足任意重排下的结合律。分块、并行归约以及融合乘加指令可能改变舍入位置或累加顺序，因此数值核验应同时检查布局、边界和合理的浮点误差，不能把所有非零差异都判定为索引错误。

5 工程规模与综合例题

5.1 两层卷积的形状与运算规模

一个用于服饰图像数据集 Fashion MNIST 分类的修改版 LeNet-5 网络，在 Mini-DNN 这一 C++ 神经网络框架中实现，优化对象是使用 CUDA 执行的卷积层前向计算。两次卷积调用分别使用 86×86 和 40×40 的输入空间尺寸，批次包含 100 至 10 000 幅图像，卷积核均为 7×7。这些尺寸指实际传给相应卷积层的张量空间大小。

表4使用步幅 1、无填充的定义，列出每幅图像的形状与计算量。参数统计只包含卷积权重，运算统计只包含卷积乘加，均不包含偏置、激活或池化。

表 4 两次卷积调用的单幅图像规模

项目	第一次调用	第二次调用
输入形状 $C\times H\times W$	$1\times86\times86$	$4\times40\times40$
输出通道数 M	4	16
核大小 $K\times K$	7×7	7×7
输出形状 $M\times H_o\times W_o$	$4\times80\times80$	$16\times34\times34$
卷积权重数 MCK^2	196	3 136
输出元素数 MH_oW_o	25 600	18 496
每个输出的乘加数 CK^2	49	196
每幅图像的乘加数	1 254 400	3 625 216
每幅图像的浮点运算数	2 508 800	7 250 432

第一次卷积的空间输出为 $86-7+1=80$，第二次为 $40-7+1=34$。如果两次卷积后均采用 2×2、步幅 2 的池化，则空间尺寸分别从 80 降到 40、从 34 降到 17；这个条件下，第一层池化输出的空间尺寸与第二次卷积输入相容。卷积本身的尺寸和计算量不依赖是否随后采用这一池化配置。

第二次调用的输出元素数仅为第一次的 $18\,496/25\,600=0.7225$，但每个输出需要处理四倍的输入通道数，因此每图计算量比为

$$\frac{16\cdot34^2\cdot4\cdot7^2}{4\cdot80^2\cdot1\cdot7^2}=0.7225\times4=2.89. \tag{27}$$

空间分辨率下降并不保证下一层计算量下降；输入通道数、输出通道数和窗口大小共同决定总量。两次卷积每图合计 4 879 616 次乘加，即 9 759 232 FLOPs。批次为 B 时，这些计算量乘以 B，卷积权重数量则保持不变。

5.2 展开工作区与优化评估

将上述两层改写为矩阵乘法时，第一次的 $D = 49, L = 6400$，第二次的 $D = 196, L = 1156$。表5给出显式展开的空间需求，其中单精度浮点格式（FP32）中每个浮点数占 4 字节。

表 5 显式展开数组的空间需求

项目	第一次调用	第二次调用
每幅图像的展开矩阵 $D \times L$	49×6400	196×1156
每幅图像的展开元素数	313 600	226 576
每幅图像的 FP32 展开字节数	1 254 400	906 304
展开数组与原输入的大小比	≈ 42.40	35.4025
$B = 100$ 的完整展开字节数	125 440 000	90 630 400
$B = 10\,000$ 的完整展开字节数	12 544 000 000	9 063 040 000

采用十进制单位 $1\,\mathrm{GB} = 10^9$ 字节时，一次展开全部 10 000 幅图像，两个调用分别需要约 12.544 GB 和 9.06304 GB 的展开工作区。这些数字仅包含展开数组，不包括输入、权重、输出及其他临时数组。两层按顺序执行并复用工作区时，容量只需覆盖其中较大的需求，不必将二者相加。

完整批次展开是一种执行选择。逐图展开、相乘并复用缓冲区时，工作区可缩小为单幅图像的 $4DL$ 字节；每次处理 B_c 幅图像的小批次时，则为 $4B_cDL$ 字节。小批次降低峰值存储占用，但可能增加执行阶段数量，并改变矩阵乘法的规模与并行度。隐式展开进一步移除了完整展开数组，直接从原输入装入当前计算块。工作区约束因此会影响算法组织，而不仅是最后的内存分配。

实现优化可以按相互依赖的三个技术阶段组织：先建立 CPU 参考结果和 GPU 基础卷积，再通过性能分析定位主要开销并实现矩阵展开及其融合，最后改进卷积与展开中的分块、寻址和存储复用。性能分析（profiling）需要区分实际执行时间、访存开销和资源占用，才能判断某项改动解决了哪一类瓶颈。

比较显式与隐式展开时，应统计完成同一卷积结果所需的全部设备计算阶段。显式方案至少包含展开和矩阵乘法；若另需布局重排，也应计入。只记录矩阵乘法内核的时间，会遗漏显式方案生成输入矩阵的成本。包含或排除主机与设备之间的数据传输也应保持一致。由于内核启动是异步的，计时终点需要对应设备工作实际完成的位置。

正确性核验应覆盖不同批次、多个输入与输出通道、非方形空间尺寸以及不能被线程块尺寸整除的边缘区域。可以先用小整数输入核对展开矩阵和输出的确切值，再用浮点输入比较不同实现的结果。每次改变线程分工、批次布局或求和组织后，都需要重新确认对应的索引关系；性能提升只有在保持预期输出和统计范围一致时才有意义。

5.3 三维卷积的存储读取比

考虑步幅为 1 的三维卷积，核的三个维度为 $5\times5\times7$，权重存于常量内存。一个线程块负责 $16\times16\times16$ 的逻辑输出块，并先将完整的联合输入区域装入共享内存，其中每个输入元素只从全局内存读取一次。卷积阶段，对每个输出的每个核位置执行一次共享内存标量读取。要求内部输出块的全局读取次数与共享读取次数之比，忽略权重访问和输出写入。

设输出块的三个维度为 T_x, T_y, T_z，卷积核的对应维度为 K_x, K_y, K_z。沿每个轴，相邻输出窗口的联合范围都比输出范围多出 $K_i - 1$ 个输入位置。因此，装入共享内存的联合输入区域包含

$$G = (T_x + K_x - 1)(T_y + K_y - 1)(T_z + K_z - 1) \tag{28}$$

个元素。每个元素只装入一次，故这也是全局读取次数。一个输出需要读取 $K_xK_yK_z$ 个共享输入，输出总数为 $T_xT_yT_z$，所以共享读取次数为

$$S = T_xT_yT_zK_xK_yK_z. \tag{29}$$

代入本题参数，输入区域尺寸为 $(16+4)\times(16+4)\times(16+6) = 20\times20\times22$，故 $G = 8\,800$。逻辑输出数为 $16^3 = 4\,096$，每个输出读取 $5\times5\times7 = 175$ 个共享输入，故 $S = 4\,096\times175 = 716\,800$。所求比值为

$$G : S = 8\,800 : 716\,800 = 11 : 896 \approx \boxed{1 : 81.45}. \tag{30}$$

这里比较的是规定计算方式下的标量读取次数。共享内存读取具有不同于全局内存读取的成本，实际实现还包含同步、地址计算和算术操作，因此该比值不是运行时间的加速比。

逻辑输出块的尺寸也不等于线程块必须采用的尺寸。16^3 个输出可以由较少线程分担，一个线程计算多个输出；实际线程块必须满足设备的线程数和资源限制。输出的分工方式应与上面采用的读取计数模型保持一致。

同样的方法适用于长方体输出块。例如，若第二个输出维度为 161，即输出块为 $16\times161\times16$，输入区域变为 $20\times165\times22$。此时

$$\begin{aligned} G &= 72\,600, \\ S &= 16\cdot161\cdot16\cdot175 = 7\,212\,800, \\ G : S &= 72\,600 : 7\,212\,800 \approx 1 : 99.35. \end{aligned} \tag{31}$$

输出块的具体尺寸决定了晕区开销，不能在读取比计算中省略任一轴的长度。

5.4 多通道卷积的浮点运算总数

一幅 100×200 的彩色图像含 3 个输入通道。卷积层使用 9×9 核生成 10 个输出特征图，每个输出特征图都结合全部 3 个输入通道，步幅为 1，不进行填充。一次乘加计为 2 次浮点运算，仅统计卷积，忽略偏置、激活、池化和其他操作。

首先计算有效输出范围。高度方向的窗口左上角可从 0 移动到 $100 - 9 = 91$，因此共有 92 个位置；宽度方向可从 0 移动到 $200 - 9 = 191$，因此共有 192 个位置。每个输出特征图有 $92 \times 192 = 17\,664$ 个值，全部十个特征图共有 176 640 个输出值。

每个输出值分别与三个通道中的 9×9 窗口做乘积并累加，需要 $3 \times 9 \times 9 = 243$ 次乘加。总乘加数为

$$N_{\mathrm{MAC}} = 10 \cdot 92 \cdot 192 \cdot 3 \cdot 9^2 = 42\,923\,520. \tag{32}$$

按一次乘加计 2 FLOPs 的约定，总浮点运算数为

$$N_{\mathrm{FLOP}} = 2 \cdot 42\,923\,520 = \boxed{85\,847\,040\ \mathrm{FLOPs}}. \tag{33}$$

若错用输入的 100×200 个空间位置，就会把不属于有效卷积输出的边界位置也计入；若漏掉输入通道因子 3，则只计算了一个输入通道的贡献。输出通道数 M 决定需要多少组核，输入通道数 C 决定每个输出内积的长度，这两个因子承担不同作用。

该层只有 $10 \times 3 \times 9^2 = 2\,430$ 个卷积权重，却执行了四千多万次乘加，因为每个权重都在 92×192 个输出位置复用。权重共享降低参数量，同时保留了对所有输出位置逐一完成计算的要求。

5.5 计算组织的统一关系

卷积的数学依赖可以分为两组索引：(b, m, h, w) 确定独立输出，(c, p, q) 确定该输出内部的乘加归约。直接实现保留这组张量索引；矩阵实现把 (c, p, q) 合并为 r，把 (h, w) 合并为 ℓ。两种表示的核心量对应为

$$D = CK^2, \qquad L = H_{\mathrm{o}} W_{\mathrm{o}}, \qquad N_{\mathrm{output}} = BML, \qquad N_{\mathrm{MAC}} = BMDL. \tag{34}$$

共享内存复用相邻输出的重叠输入，矩阵分块沿输出通道复用输入，寄存器子块在单个线程内复用操作数。显式与隐式展开保持上述索引和输出含义；融合卷积、池化、偏置与激活时，还必须保持各阶段的运算次序、布局和同步条件。